

Skillab

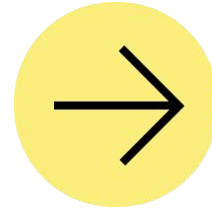
AI Agent Development

Richard Șerban Francu

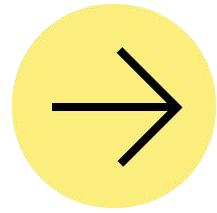
Senior AI Engineer @UCE Systems



Programa cursului



1. Noțiuni de bază — API-uri LLM și primul agent
2. Instrumente, prompturi și observabilitate
3. Procesarea și extragerea documentelor
- 4. Stocarea bazelor de date și RAG**
5. LangGraph — Stare și fluxuri de lucru
6. Orchestrarea multi-agent
7. Optimizarea ML — Dincolo de apelurile LLM
8. Memorie și cache
9. Securitate și măsuri de protecție
10. Integrarea MCP
11. Inferența SageMaker
12. Training, Fine-Tuning și RLHF



**Stocarea bazelor de
date și RAG**

Lecția 4

Stocarea bazelor de date și RAG

- Configurarea PostgreSQL pentru stocarea datelor agentului
- Proiectarea schemelor pentru datele extrase
- Funcționarea RAG (Retrieval-Augmented Generation)
- Generarea și stocarea embedding-urilor
- Utilizarea pgvector pentru căutarea similitudinilor

PostgreSQL Setup & SQLAlchemy Basics

- De ce PostgreSQL pentru AI agents
- Docker setup cu pgvector
- SQLAlchemy: Engine, Session, Base
- Document model & Alembic migrations

De ce PostgreSQL?

Database	Avantaje	Dezavantaje	Use Case
SQLite	Simplu, fără server, bun pt dev	Single-user, fără scalare	Prototyping, demo-uri
MongoDB	Schema flexibilă, JSON native	Weak consistency, relații slabe	Logs, unstructured data
PostgreSQL ✓	ACID, relații, pgvector, production	Necesită server	AI agents în producție

De ce PostgreSQL pentru Document Analyst?



ACID transactions

Extracția e atomică — sau merge tot, sau nimic



Relații clare

Document → Chunks → Embeddings



pgvector extension

Similarity search nativ în DB, fără servicii externe



JSON support

Metadata flexibilă cu JSONB columns



Production-ready

Scalabil, robust, AWS RDS support

Docker Setup — docker-compose.yml

ankane/pgvector

PostgreSQL + pgvector pre-instalat

healthcheck

Verifică că DB e gata înainte de conexiuni

volumes

Date persistente între restart-uri

ports: 5432

Portul standard PostgreSQL

```
1  services:
2    db:
3      image: pgvector/pgvector:pg16
4      restart: unless-stopped
5      environment:
6        POSTGRES_USER: ${POSTGRES_USER}
7        POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
8        POSTGRES_DB: ${POSTGRES_DB}
9      ports:
10       - "127.0.0.1:5432:5432"
11      volumes:
12       - pgdata:/var/lib/postgresql/data
13      shm_size: 256mb
14      healthcheck:
15        test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
16        interval: 10s
17        timeout: 5s
18        retries: 5
19        start_period: 30s
20      deploy:
21        resources:
22          limits:
23            memory: 1G
24
25  volumes:
26    pgdata:
```

Comenzi Docker — PostgreSQL

```
docker compose up -d
```

Pornește PostgreSQL în background

```
docker compose ps
```

Verifică că containerul rulează

```
docker compose logs postgres
```

Vezi log-urile (debug)

```
docker exec -it document_analyst_db psql ...
```

Conectare directă la DB

```
docker compose down -v
```

Stop + șterge volumele (reset complet)

SQLAlchemy — Concepte Cheie

Engine

Conexiunea la baza de date.
Creată o singură dată la startup.

SessionLocal

Factory pentru sesiuni.
Fiecare request primește propriul session.

Base

Clasa părinte pentru modele ORM.
Toate modelele moștenesc din Base.

get_db()

Pattern dependency injection.
Asigură că session-ul se închide automat.

database.py — SQLAlchemy Setup

```
1  import os
2  from dotenv import load_dotenv
3  from sqlalchemy import create_engine
4  from sqlalchemy.orm import sessionmaker, declarative_base
5
6  load_dotenv()
7
8  DATABASE_URL = os.getenv("DATABASE_URL")
9
10 # echo=True doar pentru debugging – în producție: echo=False
11 engine = create_engine(DATABASE_URL, echo=True, pool_pre_ping=True)
12
13 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
14
15 Base = declarative_base()
16
17
18 def get_db():
19     db = SessionLocal()
20     try:
21         yield db
22     finally:
23         db.close()
24
```

`load_dotenv()`

Citește .env în environment

`create_engine()`

echo=True pt debugging

`sessionmaker()`

autocommit=False e important

`get_db()`

yield → session se închide automat

models.py — Document Model



```
1 from sqlalchemy import Column, Integer, String, Text, DateTime, func
2 from sqlalchemy.dialects.postgresql import JSONB
3
4 from database import Base
5
6
7 class Document(Base):
8     __tablename__ = "documents"
9
10    id = Column(Integer, primary_key=True, autoincrement=True)
11    filename = Column(String(255), nullable=False, index=True)
12    content = Column(Text, nullable=False)
13    # "metadata" e rezervat în SQLAlchemy → atribut redenumit, coloană păstrată
14    doc_metadata = Column("metadata", JSONB, nullable=False, default=dict)
15    created_at = Column(DateTime(timezone=True), server_default=func.now(), nullable=False)
16
```

id

Primary key, auto-increment

filename

Index pt căutare rapidă

content

Full text (înainte de chunking)

metadata

JSONB — flexibil și query-able

created_at

Audit trail automat

Alembic — De ce Migrations?

✗ Fără migrations

```
create_all(engine)
→ Drop all tables? Pierzi datele!
→ ALTER manual? Error-prone
→ Schema netrackată în Git
```



✓ Cu Alembic

- Version control pt schema DB
- Migrations incrementale v1→v2→v3
- Rollback dacă ceva merge prost
- Sync dev / staging / prod

`alembic init`



`env.py config`



`autogenerate`



`upgrade head`

Workflow Alembic

Alembic — Comenzi & Migration Lifecycle

```
alembic revision --autogenerate -m "..."
```

Generează migration din modele

generate

```
alembic upgrade head
```

Aplică toate migrations

apply

```
alembic downgrade -1
```

Rollback un pas

rollback

```
alembic current
```

Versiunea curentă a DB

inspect

⚠️ Adaugă toate modelele noi în alembic/env.py → from models import Document, ...

CRUD Operations & Database Patterns

- Repository Pattern
- CRUD: Create, Read, Update, Delete
- Advanced queries & JSONB filtering
- Transaction management
- Error handling cu custom exceptions

Problema: DB Logic Împrăștiat

✗ Fără Repository

```
1  # ✗ Fără Repository – DB logic împrăștiat, duplicat
2
3  # agent.py
4  db = SessionLocal()
5  doc = db.query(Document).filter(Document.filename == "a.pdf").first()
6  db.close()
7
8  # search.py
9  db = SessionLocal()
10 docs = db.query(Document).all()
11 db.close()
12
```

✓ Cu Repository

```
1  # ✓ Cu Repository – un singur API, consistent
2
3  # agent.py
4  with transaction() as db:
5      repo = DocumentRepository(db)
6      doc = repo.get_by_filename("a.pdf")
7
8  # search.py
9  with transaction() as db:
10     repo = DocumentRepository(db)
11     docs = repo.get_all()
12
```

Repository Pattern = toată logica DB într-o singură clasă, separată de business logic

Repository Pattern — De ce?



Encapsulare

Toată logica DB într-o singură clasă.
Agent-ul nu știe că folosești SQLAlchemy.



Testabilitate

Mock repository-ul în teste.
Nu ai nevoie de DB real pentru unit tests.



Abstracție

Poți schimba PostgreSQL cu alt DB
fără să modifichi codul agentului.



Reutilizabilitate

O singură sursă de adevăr.
Fără queries duplicate în codebase.

DocumentRepository — Structură



```
1 class DocumentRepository:
2     def __init__(self, db: Session) -> None:
3         self.db = db
4
5     # CREATE
6     def create(self, filename: str, content: str, metadata: dict[str, Any]) -> Document: ...
7     def create_batch(self, documents: list[dict[str, Any]]) -> list[Document]: ...
8
9     # READ
10    def get_by_id(self, doc_id: int) -> Document | None: ...
11    def get_all(self, limit: int = 100, offset: int = 0) -> list[Document]: ...
12
13    # UPDATE
14    def update(self, doc_id: int, **fields: Any) -> Document | None: ...
15    def update_metadata(self, doc_id: int, metadata: dict[str, Any]) -> Document | None: ...
16
17    # DELETE
18    def delete(self, doc_id: int) -> bool: ...
19    def delete_batch(self, doc_ids: list[int]) -> int: ...
20
```

CREATE

```
def create()
```

```
def create_batch()
```

READ

```
def get_by_id()
```

```
def get_all()
```

UPDATE

```
def update()
```

```
def update_metadata()
```

DELETE

```
def delete()
```

```
def delete_batch()
```

CREATE — create() & create_batch()



```
1 def create(self, filename: str, content: str, metadata: dict[str, Any]) -> Document:
2     doc = Document(filename=filename, content=content, doc_metadata=metadata) # 1. creează obiect
3     self.db.add(doc) # 2. adaugă în session (nu în DB încă)
4     self.db.commit() # 3. scrie în DB
5     self.db.refresh(doc) # 4. citește ID-ul generat înapoi
6     return doc
7
8 def create_batch(self, documents: list[dict[str, Any]]) -> list[Document]:
9     docs = [Document(**d) for d in documents]
10    self.db.add_all(docs) # un singur INSERT batch, mult mai rapid decât N add()
11    self.db.commit()
12    for doc in docs:
13        self.db.refresh(doc)
14    return docs
15
```

1

Creează obiect

Document(**kwargs)
setează câmpurile

2

db.add()

Adaugă în session
(nu în DB încă)

3

db.commit()

Scrie în DB
tranzacție completă

4

db.refresh()

Citește ID-ul generat
înapoi din DB

READ — Queries de Bază



```
1 def get_by_id(self, doc_id: int) -> Document | None:
2     # API-ul modern SQLAlchemy: Session.get() folosește direct PK-ul
3     return self.db.get(Document, doc_id)
4
5 def get_by_filename(self, filename: str) -> Document | None:
6     return self.db.query(Document).filter(Document.filename == filename).first()
7
8 def get_all(self, skip: int = 0, limit: int = 100) -> list[Document]:
9     return (
10         self.db.query(Document)
11         .order_by(Document.created_at.desc())
12         .offset(skip)
13         .limit(limit)
14         .all()
15     )
16
17 def count(self) -> int:
18     return self.db.query(func.count(Document.id)).scalar()
19
```

`get_by_id(id)`

Caută după PK
Returnează None dacă nu există

`get_by_filename(f)`

Folosește index-ul
Query $O(\log n)$

`get_all(skip, limit)`

Pagination built-in
Ordonat după created_at DESC

`count()`

Total records
Util pentru UI pagination

Full-Text Search în PostgreSQL

Concepte cheie din tsvector

tsvector

Reprezentare pre-procesată a textului. Stochează lexeme și pozițiile lor, nu textul brut.

tsquery

Expresia de căutare, procesată cu același dicționar. Operatori: & (AND), | (OR), ! (NOT).

@@

Operatorul de match. Verifică dacă un tsvector se potrivește cu un tsquery.

Stemming

Reducerea la rădăcină prin dicționar de limbă. „legile” devine „lege”.

GIN Index

Inverted index: pentru fiecare lexem păstrează lista documentelor. Scalează la milioane.

ts_rank

Scor de relevanță între tsvector și tsquery. Folosit în ORDER BY pentru sortare.

setweight

Ponderi A / B / C / D pe secțiuni (titlu > conținut). ts_rank le ia în calcul automat.

ts_headline

Snippet din document cu termenii evidențiați. Preview Google-style pentru UI.

GENERATED STORED

Coloana search_vector se actualizează automat. Zero cod, zero trigger manual.

READ Avansat

```
1
2 def filter_by_metadata(self, key: str, value: str) -> list[Document]:
3     # operatorul ->> din PostgreSQL: extrage field din JSONB ca text
4     return (
5         self.db.query(Document)
6         .filter(Document.doc_metadata[key].astext == value)
7         .all()
8     )
9
10 def search(self, query: str, limit: int = 20) -> list[Document]:
11     # tsquery construit cu același dicționar ca search_vector → stemming consistent
12     ts_query = func.plainto_tsquery("romanian", query)
13     return (
14         self.db.query(Document)
15         .filter(Document.search_vector.op("@@")(ts_query))
16         .order_by(func.ts_rank(Document.search_vector, ts_query).desc())
17         .limit(limit)
18         .all()
19     )
20
```


UPDATE & DELETE

```
1 def update(self, doc_id: int, **fields: Any) -> Document | None:
2     clean = {k: v for k, v in fields.items() if k in self._UPDATABLE}
3     if not clean:
4         return self.db.get(Document, doc_id)
5     self.db.execute(sql_update(Document).where(Document.id == doc_id).values(**clean))
6     self.db.commit()
7     return self.db.get(Document, doc_id)
8
9 def update_metadata(self, doc_id: int, metadata: dict[str, Any]) -> Document | None:
10    # JSONB merge la nivel de DB (atomic) – operatorul || din PostgreSQL
11    self.db.execute(
12        sql_update(Document)
13        .where(Document.id == doc_id)
14        .values(doc_metadata=Document.doc_metadata.op("||")(metadata))
15    )
16    self.db.commit()
17    return self.db.get(Document, doc_id)
18
19 # DELETE
20 def delete(self, doc_id: int) -> bool:
21     result = self.db.execute(sql_delete(Document).where(Document.id == doc_id))
22     self.db.commit()
23     return result.rowcount > 0
24
25 def delete_batch(self, doc_ids: list[int]) -> int:
26     if not doc_ids:
27         return 0
28     result = self.db.execute(sql_delete(Document).where(Document.id.in_(doc_ids)))
29     self.db.commit()
30     return result.rowcount
31
```

UPDATE

`update(id, dict)`

Actualizeaza câmpurile din dict

`update_metadata(id, dict)`

Merge metadata, nu înlocuiește

DELETE

`delete(id)`

Șterge un document după ID

`delete_batch(ids)`

Șterge multiple documente atomic

Transaction Management

✗ Manual — error-prone

```
1  # ✗ Manual – error-prone
2
3  db = SessionLocal()
4  try:
5      doc1 = repo.create(...)
6      doc2 = repo.create(...)
7      db.commit()
8  except Exception:
9      db.rollback()    # ușor de uitat
10     raise
11 finally:
12     db.close()       # ușor de uitat
13
```



✓ Context Manager

```
1  # ✓ Context Manager – commit / rollback / close automate
2
3  with transaction() as db:
4      repo = DocumentRepository(db)
5      doc1 = repo.create(...)
6      doc2 = repo.create(...)
7      # commit → automat la ieșire fără excepție
8      # rollback → automat dacă urcă o excepție
9      # close → garantat, întotdeauna
10
```

Commit automat

Rollback automat

Close garantat

transaction() — Context Manager



```
1  from contextlib import contextmanager
2  from typing import Generator
3  from sqlalchemy.orm import Session
4
5  from database import SessionLocal
6
7
8  @contextmanager
9  def transaction() -> Generator[Session, None, None]:
10     db = SessionLocal()          # 1. creează session nou
11     try:
12         yield db                 # 2. execută codul din with-block
13         db.commit()              # 3. totul OK → scrie în DB
14     except Exception:
15         db.rollback()            # 4. excepție → anulează tranzacția
16         raise
17     finally:
18         db.close()               # 5. mereu – eliberează conexiunea
19
```

`SessionLocal()`

Creează session nou



`yield db`

Execută codul din with block



`db.commit()`

Dacă totul OK → scrie în DB



`db.rollback()`

Dacă Exception → anulează



`db.close()`

Finally → mereu se execută

Error Handling — Custom Exceptions

```
1 class DocumentError(Exception):
2     """Bază pentru toate erorile din domeniul Document."""
3
4
5 class DocumentNotFoundError(DocumentError):
6     def __init__(self, doc_id: int) -> None:
7         super().__init__(f"Document with id={doc_id} not found")
8         self.doc_id = doc_id
9
10
11 class DuplicateDocumentError(DocumentError):
12     def __init__(self, filename: str) -> None:
13         super().__init__(f"Document with filename='{filename}' already exists")
14         self.filename = filename
15
16
17 class InvalidMetadataError(DocumentError):
18     def __init__(self, reason: str) -> None:
19         super().__init__(f"Invalid metadata: {reason}")
20         self.reason = reason
21
22
```

DocumentNotFoundError

ID nu există în DB
get_by_id_safe() o ridică

DuplicateDocumentError

Filename deja există
create_safe() o ridică

InvalidMetadataError

Format metadata greșit
validare la update

Best Practices

1. Folosește mereu Repository Pattern



```
1 with transaction() as db:
2     repo = DocumentRepository(db)
3     doc = repo.get_by_filename("a.pdf")
4
```



```
1 db = SessionLocal()
2 doc = db.query(Document).filter(Document.filename == "a.pdf").first()
3
4
```

2. Session Management — context manager



```
1 # ✓
2 with transaction() as db:
3     repo = DocumentRepository(db)
4     # commit / rollback / close → automate
5
```



```
1 # ✗
2 db = SessionLocal()
3 # uiți db.close() → connection leak
4
```

3. Pagination mereu cu total count



```
1
2 # ✓
3 results, total = repo.get_all(skip=0, limit=20)
4 # returnezi (results, total) → UI știe câte pagini sunt
5
```



```
1 # ✗
2 docs = repo.get_all()
3 # fără total → UI nu poate calcula numărul de pagini
4
```

Lecția 04

Extractor Agent Database Integration

- Pydantic schemas pentru validare
- ExtractorAgentV2: load → extract → save
- Batch processing cu error handling
- Unit testing cu Mock LLM

v1 → v2: De ce avem nevoie de DB?

✗ Extractor v1 (Lecția 3)



✓ Extractor v2 (această secțiune)



Query documente

Analytics

RAG ready

Flow Detaliat — Extractor v2



1. Load

Citește fișierul (PDF, DOCX, TXT)
Extrage textul complet



2. Extract

LLM extrage metadata structurată
title, doc_type, author, date



3. Validate

Pydantic validează schema
Field validators, type checking



4. Save DB

DocumentRepository.create()
Salvează în PostgreSQL



5. Error Handling

Log eroarea, skip documentul
Batch continuă cu celelalte

Pydantic Schemas — Validare înainte de Save

```
1 @field_validator("content")
2 @classmethod
3 def content_not_empty(cls, v: str) -> str:
4     v = v.strip()
5     if not v:
6         raise ValueError("content cannot be empty or whitespace-only")
7     return v
8
9 @field_validator("doc_type", mode="before")
10 @classmethod
11 def validate_doc_type(cls, v: Any) -> str:
12     # valori necunoscute -> 'other' (în loc să crape validarea)
13     allowed = {"invoice", "contract", "report", "letter", "other"}
14     return v if v in allowed else "other"
15
```

```
1 class ExtractionResult(BaseModel):
2     success: bool
3     document: DocumentExtraction | None = None
4     error: str | None = None
5
```

filename

Fișierul original

doc_type

Validat: invoice/contract/...

to_db_format()

Convertește pt repo.create()

content

Text complet, min_length=1

extra_metadata

Dict flexibil pentru orice

Required

Optional

@field_validator

content_not_empty
→ strip() + check whitespace
validate_doc_type
→ default 'other' dacă necunoscut

ExtractorAgentV2 — Structură

```
1
2 class ExtractorAgentV2:
3     def __init__(self, model: str = "gemini-2.0-flash") -> None:
4         self.model = model
5         self._client: genai.Client | None = None # lazy
6
7     @property
8     def client(self) -> genai.Client:
9         # init per proces - clientul nu e pickle-able
10        if self._client is None:
11            self._client = genai.Client()
12        return self._client
13
14    def load_document(self, path: str | Path) -> str | None:
15        # existing din Lecția 3, recap pe slide 33 (cu error handling adăugat)
16        ...
17
18    def extract_metadata(self, content: str, filename: str) -> DocumentExtraction | None:
19        # LLM call cu response_schema -> Pydantic direct, fără parser intermediar
20        ...
21
22    def process_document(self, path: str | Path) -> ExtractionResult:
23        # Load -> Extract -> Validate -> Save (atomic, idempotent pe filename)
24        ...
25
26    def process_batch(self, paths: list[str | Path]) -> list[ExtractionResult]:
27        # continue-on-error: o eroare pe un doc nu oprește batch-ul
28        ...
29
```



`__init__(llm)`

PydanticOutputParser
extraction_prompt template



`load_document(path)`

PyPDFLoader / Docx2txt
/ TextLoader — auto detect



`extract_metadata()`

LLM call + parse
Pydantic output



`process_document()`

Load -> Extract -> Validate
-> Save atomic



`process_batch(paths)`

Multiple files
continue on error

process_document() — Flow Complet



```
1 def process_document(self, path: str | Path) -> ExtractionResult:
2     path = Path(path)
3
4     # 1. Load
5     content = self.load_document(path)
6     if content is None:
7         return ExtractionResult(success=False, error=f"load failed: {path.name}")
8
9     # 2. Extract + Validate (Pydantic via response_schema)
10    extraction = self.extract_metadata(content, filename=path.name)
11    if extraction is None:
12        return ExtractionResult(success=False, error=f"extract failed: {path.name}")
13
14    # 3. Save — idempotent: dacă filename există deja, skip
15    with transaction() as db:
16        repo = DocumentRepository(db)
17        existing = repo.get_by_filename(path.name)
18        if existing is not None:
19            logger.info("Skip duplicate: %s (id=%d)", path.name, existing.id)
20            return ExtractionResult(success=True, document=extraction)
21
22    doc = repo.create(**extraction.to_db_format())
23    logger.info("Saved %s (id=%d)", path.name, doc.id)
24    return ExtractionResult(success=True, document=extraction)
25
```

load_document()

None → ExtractionResult(success=False)



extract_metadata()

None → ExtractionResult(success=False)



to_db_format()

Dict pentru repo.create()



repo.create()

Document salvat în PostgreSQL



ExtractionResult

success=True, document_id=doc.id

RAG Architecture & Embeddings Introduction

- Ce este RAG și de ce îl folosim
- RAG workflow: Index → Retrieve → Generate
- Ce sunt embeddings și cum funcționează
- Sentence-Transformers setup
- Similarity search demo practic

Problema fără RAG

✗ Fără RAG

```
response = llm.invoke(  
    "Ce spune contractul despre reziliere?"  
)  
# ✗ LLM nu are contractul!  
# → Hallucinates sau "nu știu"
```

Fără RAG, LLM-ul e orb față de datele tale.

- Nu știe ce e în contractul tău
- Nu știe rapoartele interne
- Poate inventa răspunsuri false



Cutoff date

LLM știe doar până în 2023-2024
Datele noi nu există pentru el



Date private

Contractele, rapoartele interne
nu au fost în training data



Hallucinations

Generează răspunsuri plauzibile
dar false cu încredere maximă

Soluția: RAG — Retrieve → Augment → Generate



```
1
2
3 def rag_solution(question: str) -> str:
4     # 1. Retrieve – caută documentele relevante (vector search, vezi slide-urile următoare)
5     docs = retrieve_relevant_docs(question, top_k=3)
6
7     # 2. Augment – adaugă contextul în prompt
8     context = "\n\n".join(d.content for d in docs)
9     prompt = f"""Răspunde pe baza contextului de mai jos. Dacă răspunsul nu e în context, spune-o explicit.
10
11 Context:
12 {context}
13
14 Întrebare: {question}
15 """
16
17     # 3. Generate – LLM-ul răspunde cu context
18     response = client.models.generate_content(model="gemini-2.0-flash", contents=prompt)
19     return response.text
20
```



Actualizat

Adaugi doc-uri noi fără re-antrenare LLM



Date private

Documente confidențiale in-house



Fără hallucinations

Răspunsuri bazate pe fapte reale



Citable

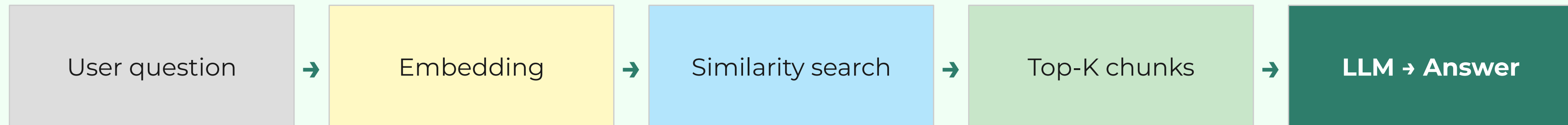
Poți arăta sursa fiecărui răspuns

RAG Workflow — Două Faze

OFFLINE — INDEX (se face o dată)



ONLINE — QUERY (fiecare întrebare)



Ce sunt Embeddings?

Embedding = vector de numere care reprezintă semantic meaning-ul unui text

"pisica doarme pe canapea"



[0.20, -0.50, 0.80, 0.10, ...]

Similar cu cel de jos ↓

"motanul se odihnește pe sofa"



[0.19, -0.48, 0.82, 0.09, ...]

Aproape identic! ↑

"mașina merge repede"



[-0.60, 0.30, -0.20, 0.70, ...]

Complet diferit ↕

Texte cu meaning similar → embeddings similare (vectori apropiați în spațiu)

Cosine Similarity — Cum Măsurăm Similaritatea

Cosine Similarity

Măsoară unghiul dintre doi vectori.
0 = complet diferit | 1 = identic

- 0.85+ Paraphrase / același sens
- 0.50–0.84 Topic similar
- 0.20–0.49 Oarecum legat
- < 0.20 Complet diferit

Exemplu — Similarity Matrix:

	pisica pe canapea	motanul pe sofa	mașina merge	contractul reziliere
pisica pe canapea	—	0.847 ✓	0.156	0.098
motanul pe sofa	0.847 ✓	—	0.162	0.102

Setup: HuggingFace Sentence-Transformers

```
pip install sentence-transformers
```

Model	Dimensiuni	Limbă	Use Case
all-MiniLM-L6-v2	384	EN	General, rapid, bun pt learning
paraphrase-multilingual-MiniLM-L12-v2 ★	384	Multi (RO ✓)	Documente românești — cursul nostru
all-mpnet-base-v2	768	EN	Mai precis, dar mai lent și mai mare

```
1 embeddings = model.encode(sentences, convert_to_numpy=True)
2
3 print(embeddings.shape) # (3, 384) - 3 propoziții, 384 dimensiuni
4
```


Similarity Calculation Demo



```
1 query = "pisica"
2 candidates = ["persian", "animal", "masina"]
3
4 # 1. Embedding-uri (un singur call, batch)
5 query_emb = model.encode(query, convert_to_tensor=True)
6 candidate_embs = model.encode(candidates, convert_to_tensor=True)
7
8 # 2. Cosine similarity: tensor de shape (1, len(candidates))
9 scores = util.cos_sim(query_emb, candidate_embs)[0]
10
11 # 3. Sortare descrescătoare după score, păstrăm indexul
12 ranked = sorted(enumerate(scores), key=lambda x: x[1], reverse=True)
13
14 # 4. Top-K
15 top_k = 3
16 for idx, score in ranked[:top_k]:
17     print(f"[0] {query} → vs {candidates[idx]}: {score:.3f}")
18
```

```
# Output așteptat:
```

```
[0] pisica → vs persian : 0.847 ← Very similar!
[0] pisica → vs animal: 0.432 ← Somewhat similar
[0] pisica → vs masina: 0.156 ← Not similar
```

`util.cos_sim()`

Calculează cosine similarity
între doi vectori

`model.encode()`

Text → vector 384 dimensiuni
batch sau single

`sorted(enumerate)`

Sortează rezultatele
descrescător după score

`results[:top_k]`

la primele K rezultate
Top-K retrieval

Semantic Search Demo Contract Query

```
1 clauses = [  
2     "Clauza 5.3: Rezilierea contractului se face cu preaviz de 30 de zile.",  
3     "Clauza 9.2: Litigiile se soluționează la instanțele din București.",  
4     "Clauza 7.1: Penalitățile pentru întârziere sunt de 0.1% pe zi.",  
5 ]  
6  
7 query = "Cum funcționează rezilierea?"  
8 THRESHOLD = 0.4 # sub acest scor → considerăm irelevant pentru RAG  
9  
10 query_emb = model.encode(query, convert_to_tensor=True)  
11 clause_embs = model.encode(clauses, convert_to_tensor=True)  
12  
13 scores = util.cos_sim(query_emb, clause_embs)[0]  
14 ranked = sorted(enumerate(scores), key=lambda x: x[1], reverse=True)  
15  
16 print(f"Query: '{query}')"   
17 for idx, score in ranked:  
18     flag = "✓" if score >= THRESHOLD else " "  
19     print(f"Score: {score:.3f} {flag} {clauses[idx]}")  
20
```

Output:

Query: 'Cum funcționează rezilierea?'

Score: 0.723 ✓ Clauza 5.3: Rezilierea...

Score: 0.312 Clauza 9.2: Litigiile...

Score: 0.289 Clauza 7.1: Penalitățile...

Observații cheie

0.723 Extrem de relevant

Query și doc-ul au același
subiect: reziliere

0.312 Parțial relevant

Ambele sunt clauze
dintr-un contract

0.289 Slab relevant

Context contractual
dar alt subiect

Threshold recomandat: > 0.4 pentru RAG

Proprietățile Embeddings

1. Dimensionality

```
embedding.shape # (384,)
```

384 dim = rapid, precis suficient
768 dim = mai precis, mai lent
Trade-off: precizie vs viteză

2. Normalizare

```
np.linalg.norm(emb) # ≈ 1.0
```

Vectorii sunt normalized (magnitude=1)
Facilitează cosine similarity
nu ai nevoie de scaling

3. Cross-lingual

```
sim(EN, RO) # 0.68-0.75
```

Modele multilingual înțeleg același sens în limbi diferite
RO ↔ EN funcționează!

4. Semantic vs Lexical

```
sem_sim # 0.85 lex_sim # 0.70
```

Semantic > Lexical
model-ul înțelege meaning-ul
nu doar cuvintele

RAG End-to-End — rag_query()



```
1 from sentence_transformers import SentenceTransformer, util
2 from google import genai
3
4 model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
5 client = genai.Client()
6
7
8 def rag_query(question: str, documents: list[str], top_k: int = 3) -> str:
9     # 1. Question → vector
10    q_emb = model.encode(question, convert_to_tensor=True)
11
12    # 2. Similarity cu toate doc-urile (în prod: embeddings pre-calculate în pgvector)
13    doc_embs = model.encode(documents, convert_to_tensor=True)
14    scores = util.cos_sim(q_emb, doc_embs)[0]
15
16    # 3. Top-K indecși, sortați descrescător
17    top_idx = torch.argsort(scores, descending=True)[:top_k].tolist()
18
19    # 4. Construiește contextul
20    context = "\n\n".join(documents[i] for i in top_idx)
21
22    # 5. Generează răspunsul cu context
23    prompt = f""""Răspunde pe baza contextului. Dacă răspunsul nu e în context, spune-o explicit.
24
25 Context:
26 {context}
27
28 Întrebare: {question}
29 """
30    response = client.models.generate_content(model="gemini-2.0-flash", contents=prompt)
31    return response.text
32
```

1

`model.encode(question)`

Question → vector



2

`util.cos_sim()`

Similarity cu toate doc-urile



3

`argsort(descending)[:k]`

Top-K indecși



4

`join(documents[i])`

Construiește context string



5

`llm.invoke(prompt)`

Generează răspunsul final

Când să Folosești RAG?

✓ Folosește RAG

Date private/confidențiale

contracte, rapoarte interne

Date actualizate frecvent

fără re-antrenare LLM

Precizie critică

trebuie să citezi surse

Volume mari de documente

sute/mii de doc-uri

✗ Nu folosi RAG

General knowledge

"Ce e Python?" → LLM știe deja

Creative tasks

scriere, brainstorming

Documente mici (1-2)

pune-le direct în prompt

Real-time data

weather, stock prices → API call

pgvector Setup & Similarity Search

- pgvector: vector storage în PostgreSQL
- DocumentChunk model (one-to-many)
- ChunkRepository + similarity search
- RAGService pipeline complet
- HNSW index pentru performance

Unde suntem și ce urmează

✓ Ce avem până acum

- Document salvat în PostgreSQL
- Embeddings generate cu sentence-transformers
- RAG demo în memorie (Secțiunea 4)

✗ Problema

- Embeddings sunt în memorie → se pierd
- Nu putem face similarity search persistent
- Chunks nu sunt salvate în DB

Soluția acestei secțiuni: **DocumentChunk table + pgvector extension + RAGService**



DocumentChunk

Salvează chunks
în PostgreSQL



pgvector

Vector(384) type
+ similarity ops



RAGService

Search persistent
la scară mare

Ce este pgvector?

pgvector = extensie PostgreSQL care adaugă suport nativ pentru vectori și similarity search



vector data type

```
embedding =  
Column(Vector(384))
```

Stochează arrays de floats
direct în PostgreSQL



Similarity operators

```
<=>  <->  <#>
```

Cosine / L2 / Inner Product
direct în SQL queries



Indexing

```
HNSW  /  IVFFlat
```

Search rapid $O(\log n)$
pentru milioane de vectori

pgvector vs alternative:

pgvector

Native PostgreSQL, ACID, SQL,
free

Pinecone

Managed cloud, costisitor,
extern

Weaviate

DB separat, mai complex de
operat

Chroma

In-memory/local, nu
production-ready

DocumentChunk Model — Schema

```
1  __tablename__ = "document_chunks"
2
3  id = Column(Integer, primary_key=True, autoincrement=True)
4  document_id = Column(
5      Integer,
6      ForeignKey("documents.id", ondelete="CASCADE"),
7      nullable=False,
8  )
9  content = Column(Text, nullable=False)           # 500-1000 caractere
10 chunk_index = Column(Integer, nullable=False)    # poziția în document: 0, 1, 2, ...
11 embedding = Column(Vector(384), nullable=False)  # pgvector, 384 dim
12
13 document = relationship("Document", back_populates="chunks")
14
15 __table_args__ = (
16     UniqueConstraint("document_id", "chunk_index", name="uq_doc_chunk_idx"),
17     Index("ix_chunks_document_id", "document_id"),
18 )
19
20
```

document_id

FK → documents.id
ondelete='CASCADE'

content

Textul chunk-ului
500-1000 caractere

chunk_index

Poziția în document
0, 1, 2, ...

Vector(384)

pgvector type
384 dimensiuni float

cascade=

'all, delete-orphan'

Șterge chunks
automatat cu Doc

Document → DocumentChunk: One-to-Many

Document

id: 1

filename: "contract.pdf"

content: (text lung...)

metadata: {...}

created_at: ...

1 → N
chunks

DocumentChunk [0]

content: "Clauza 1.1: Prezentul contract..."

embedding: [0.23, -0.45, 0.67, ...]

DocumentChunk [1]

content: "Clauza 5.3: Rezilierea poate..."

embedding: [0.19, -0.38, 0.72, ...]

DocumentChunk [2]

content: "Clauza 7.1: Penalitățile sunt..."

embedding: [-0.12, 0.55, 0.34, ...]

cascade="all, delete-orphan" → dacă ștergi Document, se șterg automat toate chunks-urile lui

Migration — Add DocumentChunk Table

```
alembic revision --autogenerate  
-m "add document_chunks table"
```

generate

CREATE EXTENSION

Activează pgvector
automated în migration

```
alembic upgrade head
```

apply

Vector(384)

Import din
pgvector.sqlalchemy

```
\d document_chunks
```

verify

ForeignKeyConstraint

ondelete='CASCADE'
auto-delete chunks

```
# \d document_chunks - output:
```

id	integer	not null
document_id	integer	not null
content	text	not null
chunk_index	integer	not null
embedding	vector(384)	

op.create_index()

Index pe document_id
pentru queries rapide

ChunkRepository — CRUD + Similarity Search

```
1
2 class ChunkRepository:
3     def __init__(self, db: Session) -> None:
4         self.db = db
5
6     # WRITE
7     def create_chunk(
8         self,
9         document_id: int,
10        content: str,
11        chunk_index: int,
12        embedding: list[float],
13    ) -> DocumentChunk: ...
14
15    def create_chunks_batch(self, chunks: list[dict]) -> list[DocumentChunk]: ...
16
17    # READ
18    def get_document_chunks(self, document_id: int) -> list[DocumentChunk]: ...
19    def get_chunk_by_id(self, chunk_id: int) -> DocumentChunk | None: ...
20
21    # SEARCH
22    def similarity_search(
23        self,
24        query_embedding: list[float],
25        top_k: int = 5,
26    ) -> list[tuple[DocumentChunk, float]]: ...
27
28    def search_with_threshold(
29        self,
30        query_embedding: list[float],
31        min_score: float = 0.4,
32        top_k: int = 5,
33    ) -> list[tuple[DocumentChunk, float]]: ...
34
```

WRITE

`create_chunk()`

Single chunk + embedding

`create_chunks_batch()`

Multiple chunks atomic

READ

`get_document_chunks()`

Toate chunks unui doc

`get_chunk_by_id()`

Un chunk după ID

SEARCH

`similarity_search()`

Top-K nearest chunks

`search_with_threshold`

Similarity Search — Operatorul $\lt=>$ pgvector



Cosine Distance

Recomandat pt text
embeddings normalized



L2 Distance

Distanță euclidiană
pentru vectori raw



Inner Product

Dot product
normalized = cosine



```
1 def similarity_search(  
2     self,  
3     query_embedding: list[float],  
4     top_k: int = 5,  
5 ) -> list[tuple[DocumentChunk, float]]:  
6     # 1 - (emb <=> q) -> cosine distance convertit în similaritate [0, 1]  
7     similarity = (1 - DocumentChunk.embedding.cosine_distance(query_embedding)).label("score")  
8  
9     stmt = (  
10         select(DocumentChunk, similarity)  
11         .options(joinedload(DocumentChunk.document)) # eager load - evită N+1  
12         .order_by(DocumentChunk.embedding.cosine_distance(query_embedding))  
13         .limit(top_k)  
14     )  
15  
16     rows = self.db.execute(stmt).all()  
17     return [(chunk, float(score)) for chunk, score in rows]  
18
```

1 - (emb $\lt=>$ q)
Convertim distanță
în similarity score 0-1

ORDER BY emb $\lt=>$
Sortare după
cel mai apropiat

LIMIT :top_k
Returnează top K
rezultate

.join(Document)
Eager load doc info
cu chunk-ul

RAGService — Pipeline Complet

```
1  class RAGService:
2      _model: SentenceTransformer | None = None  # singleton per proces
3
4      def __init__(self, db: Session) -> None:
5          self.repo = ChunkRepository(db)
6
7      @property
8      def model(self) -> SentenceTransformer:
9          # lazy + shared: modelul (~80MB) se încarcă o singură dată per proces
10         if RAGService._model is None:
11             RAGService._model = SentenceTransformer(MODEL_NAME)
12         return RAGService._model
13
14     def search(
15         self,
16         query: str,
17         top_k: int = 5,
18     ) -> list[tuple[DocumentChunk, float]]:
19         query_emb = self.model.encode(query, convert_to_numpy=True).tolist()
20         return self.repo.similarity_search(query_emb, top_k=top_k)
21
22     def get_context(
23         self,
24         query: str,
25         top_k: int = 5,
26         threshold: float = DEFAULT_THRESHOLD,
27     ) -> str:
28         results = self.search(query, top_k=top_k)
29         relevant = [(chunk, score) for chunk, score in results if score >= threshold]
30         if not relevant:
31             return ""
32         return "\n\n".join(
33             f"[{chunk.document.filename}] {chunk.content}" for chunk, _ in relevant
34         )
35
```

`__init__(db)`

SentenceTransformer +
ChunkRepository init

`search(query)`

encode → similarity_search
→ (chunk, score) list

`get_context()`

search() →
context string pt prompt

Flow get_context():

encode(query) → similarity_search() →
filter threshold → build context →
return string

RAG Pipeline — Usage End-to-End

```
1
2 PROMPT_TEMPLATE = """Răspunde scurt, pe baza contextului. Citează sursa în paranteză.
3 Dacă răspunsul nu e în context, spune-o explicit.
4
5 Context:
6 {context}
7
8 Întrebare: {question}
9 """
10
11
12 def ask(rag: RAGService, question: str) -> str:
13     context = rag.get_context(question)
14     if not context:
15         return "Nu am găsit informații relevante în documente."
16
17     prompt = PROMPT_TEMPLATE.format(context=context, question=question)
18     response = client.models.generate_content(model="gemini-2.0-flash", contents=prompt)
19     return response.text
20
21
22 # Usage
23 with transaction() as db:
24     rag = RAGService(db)
25
26     for q in [
27         "Cum pot rezilia?",
28         "Care sunt penalitățile?",
29         "Cât costă?",
30     ]:
31         print(f"Q: {q}")
32         print(f"A: {ask(rag, q)}\n")
33
```

Output demo:

Q: Cum pot rezilia?

A: Rezilierea se face
cu preaviz 30 zile
(Clauza 5.3)

Q: Care penalități?

A: 2% pe zi pentru
întârziere la plată
(Clauza 7.1)

Q: Cât costă?

A: 50.000 EUR în 3
tranche egale
(Clauza 2.4)

Indexing Strategies

- Linear scan vs ANN indexes
- HNSW — graph-based, rapid, mult RAM
- IVFFlat — cluster-based, puțin RAM
- Performance benchmarks comparative
- Decision tree: care index să folosești

Problema: Linear Scan $O(n)$

```
-- Fără index: scanează TOATE chunks-urile

SELECT * FROM document_chunks
ORDER BY embedding <=> '[0.1, 0.2, ...]'
LIMIT 5;

-- Complexitate:  $O(n)$  – linear!
```

Chunks	Fără Index	Status
1,000	~10-20ms	✓ OK
10,000	~100-200ms	⚠ Slow
100,000	~1-2s	✗ Inacceptabil
1,000,000	~10-20s	✗ Foarte lent

De ce e o problemă?

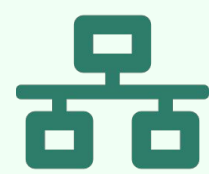
- Calculează distanța față de FIECARE vector din tabelă
- Crește linear cu numărul de chunks
- La producție avem 100k+ chunks
- Latency inacceptabilă pentru utilizatori

Soluția: ANN Index

Approximate Nearest Neighbor
 $O(\log n)$ în loc de $O(n)$

Soluția: ANN Index — Approximate Nearest Neighbor

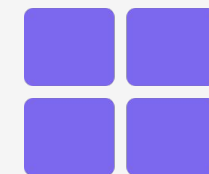
ANN = nu scanează toți vectorii, folosește structuri de date pentru approximate search



HNSW

Hierarchical Navigable
Small World

- Graph-based index
- Multi-layer hierarchy (autostrăzi → stradele)
- Cel mai rapid — $O(\log n)$
- Recall excelent (98%+)
- Necesită mult RAM (2-3x data)



IVFFlat

Inverted File
with Flat compression

- Cluster-based index
- Împarte vectorii în grupuri (lists)
- Mai lent decât HNSW
- Recall bun (92%+)
- Folosește mai puțin RAM

HNSW — Cum Funcționează



Analogia Orașului

Layer 3 (top)

Autostrăzi

Puține noduri, salturi mari între orașe

Layer 2

Bulevarde

Salturi între cartiere

Layer 1 (bottom)

Stradele mici

Navigare fină până la adresa exactă

Search: Start layer 3 → coboară → găsește vecini apropiați

Complexitate: $O(\log n)$ — logaritmică, nu liniară

HNSW — Caracteristici

✓ Avantaje

- ✓ Cel mai rapid index
- ✓ Recall excelent (98%+)
- ✓ $O(\log n)$ query time
- ✓ Scalabil la milioane de vectori

✗ Dezavantaje

- ✗ Multă memorie (2-3x data)
- ✗ Build time lung
- ✗ Update cost ridicat

HNSW Index — Performance Optimization

Fără index: Linear scan $O(n)$ → OK pentru < 10K chunks

Cu HNSW index: $O(\log n)$ → necesar pentru > 10K chunks

```
-- Create HNSW index:
```

```
CREATE INDEX ix_chunks_emb_hnsw  
ON document_chunks  
USING hnsw (embedding vector_cosine_ops)  
WITH (m = 16, ef_construction = 64);
```

```
# Benchmark (1000 chunks):
```

Fără index: ~50-200ms

Cu HNSW: ~5-20ms ← 10x mai rapid!

m = 16

Max connections per layer
Mai mare → better recall
mai mare index

ef_construction = 64

Search depth la build
Mai mare → mai precis
dar mai lent de creat

vector_cosine_ops

Operator pentru index
Match-ează $\leq$ în query
folosit pt text embeddings

HNSW — Parametri & Create Index

```
-- Create HNSW index

CREATE INDEX ix_chunks_embedding_hnsw
ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

Parametru	Default	Recomandare	Descriere
m	16	16 / 32 (high recall)	Max connections/layer. Larger = better recall, mai mult RAM
ef_construction	64	64 / 128 (high quality)	Search depth la build. Larger = better quality, mai lent build
ef_search	40	40-100	Query time param. SET la runtime

★ Recommended defaults: m=16, ef_construction=64 — echilibru bun pentru majoritatea use case-urilor

IVFFlat — Cum Funcționează



Analogia Bibliotecii

lists

Rafturi

Fiecare raft = un cluster de vectori similari

centroid

Etichetă

Centrul fiecărui cluster (raftul principal)

probes

Rafturi verificate

Câte rafturi cauți (nu le verifici pe toate!)

Search: găsește cele mai apropiate 'probes' clusters → scanează doar acelea

Nu scanează toți vectorii — doar vectorii din clusters relevante

IVFFlat — Caracteristici



Avantaje

✓ Puțin RAM (2x mai mic vs HNSW)

✓ Build rapid

✓ Update-friendly



Dezavantaje

✗ Mai lent decât HNSW

✗ Recall mai mic (92%)

✗ Tuning necesar (lists + probes)

Ideal: Development, write-heavy, RAM limitat

IVFFlat — Parametri & Create Index

```
-- Create IVFFlat index

CREATE INDEX ix_chunks_embedding_ivfflat
ON document_chunks
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

Parametru	Formula	Recomandare	Descriere
lists	rows / 1000	100 (10k-100k rows)	Nr de clusters. Mai mult, indexul este mai precis, build mai lent
probes	lists / 10	10 (runtime)	Câte clusters scanezi. Setat la query time, nu la crearea indexului

Când să folosești?

- Write-heavy workload
- RAM limitat
- Recall 90-92% acceptabil
- Development / prototyping
- Dataset relativ static

probes se setează la QUERY TIME, nu la index creation!

```
SET ivfflat.probes = 10; -- în psql sau în aplicație
```

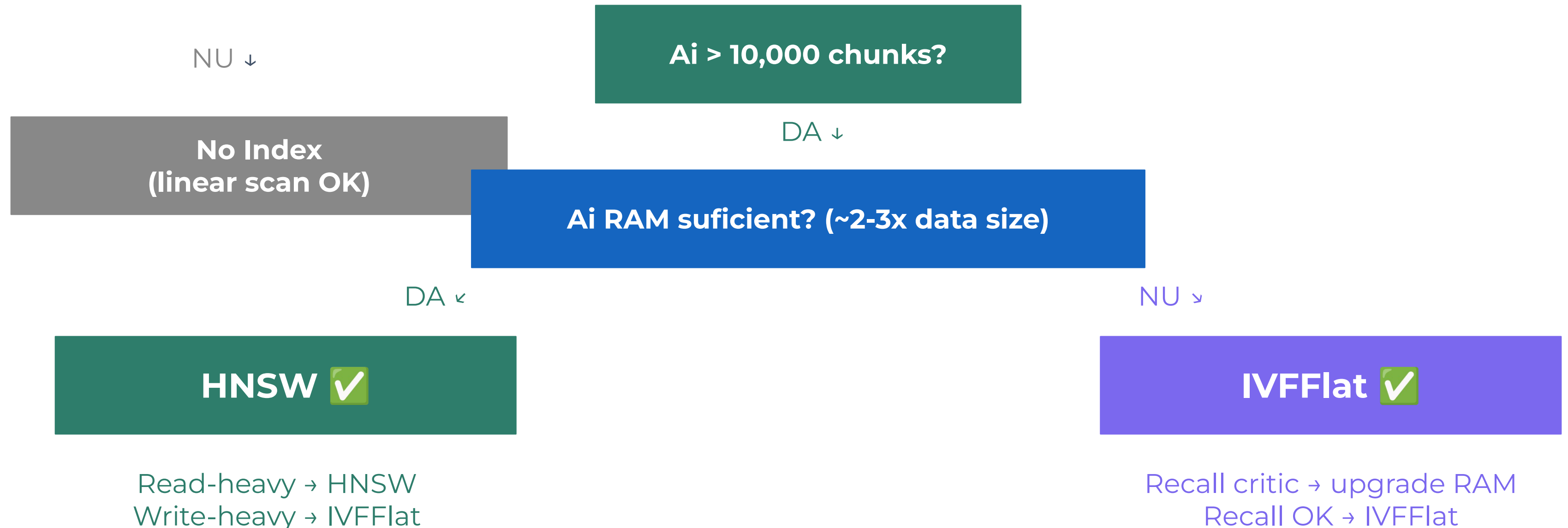

Performance Comparison — 50K Chunks

Index Type	Avg Query	Recall	Index Size	Build Time
✗ No index	850ms	100%	0 MB	0s
✓ HNSW (m=16)	12ms ⚡	98%	450 MB	180s
⚠ IVFFlat (l=100, p=10)	45ms	92%	180 MB	45s

Scalabilitate — Query Time:

Dataset	No Index	HNSW	IVFFlat
10k chunks	50ms ✓	8ms ✓	20ms ✓
100k chunks	1800ms ✗	15ms ✓	70ms ✓
1M chunks	~20s ✗	35ms ✓	350ms ⚠

Decision Tree — Care Index să Folosești?



Read-heavy → HNSW
Write-heavy → IVFFlat

Recall critic → upgrade RAM
Recall OK → IVFFlat

Recomandări practice:

Development

< 10k: No index | 10k-50k: IVFFlat

Production

50k-500k: HNSW | > 500k: HNSW + partitioning

Dacă nu ești sigur:

Start cu IVFFlat → upgrade la HNSW dacă e nevoie

Monitoring & Create Index Python

```
1
2 def create_index() -> None:
3     with engine.begin() as conn:
4         # 1. Drop dacă există (idempotent)
5         conn.execute(text(f"DROP INDEX IF EXISTS {INDEX_NAME}"))
6
7         # 2. Build HNSW cu cosine – în prod: CREATE INDEX CONCURRENTLY (nu blochează tabela)
8         conn.execute(text(f"""
9             CREATE INDEX {INDEX_NAME}
10            ON {TABLE}
11            USING hnsw ({COLUMN} vector_cosine_ops)
12            WITH (m = 16, ef_construction = 64)
13            """))
14
15        # 3. Verificare: index există și dimensiunea lui
16        result = conn.execute(text("""
17            SELECT pg_size_pretty(pg_relation_size(:idx))
18            """), {"idx": INDEX_NAME}).scalar()
19
20        logger.info("Index %s created, size=%s", INDEX_NAME, result)
21
```

```
1 SELECT
2     indexrelname AS index_name,
3     pg_size_pretty(pg_relation_size(indexrelid)) AS size
4 FROM pg_stat_user_indexes
5 WHERE relname = 'document_chunks';
```

EXPLAIN ANALYZE

Verifică dacă query folosește index-ul (Index Scan vs Seq Scan)

pg_relation_size()

Verifică dimensiunea index-ului în MB util pt planificare RAM

\timing on

Activează timing în psql pentru măsurare manuală

create_index.py

Script Python cu DROP IF EXISTS + verificare automată

Complete RAG Integration

- RAG în agent workflow — before & after
- AnalystAgent integration pattern
- Error handling: no context, low confidence
- Caching: embedding cache + lru_cache
- Observability: logging, metrics, LangSmith

Ce am construit → Ce integrăm acum

✓ Componentele RAG construite

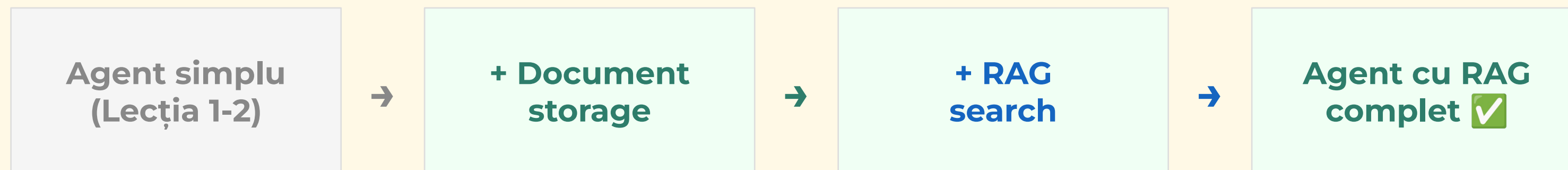
	Document storage	PostgreSQL (Secțiunile 1-3)
	Chunking & embeddings	sentence-transformers (Sec. 4-5)
	Vector search	pgvector + similarity (Sec. 5)
	Indexing	HNSW / IVFFlat (Sec. 6)



🎯 Această secțiune

- Integrarea tuturor componentelor într-un agent workflow
- Patterns pentru producție: error handling, caching, logging
- Concepte și pseudocode — codul complet furnizat separat

Transformarea agentului:



RAG în Agent Workflow | Before & After

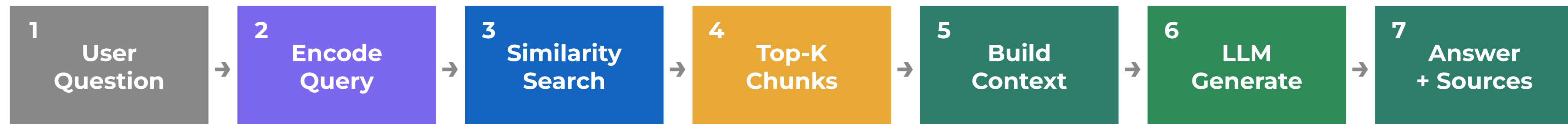
❌ Fără RAG

```
1 # ❌ Fără RAG – hallucination sau răspuns generic
2
3 question = "Ce spune contractul despre reziliere?"
4 response = client.models.generate_content(
5     model="gemini-2.0-flash",
6     contents=question,
7 )
```

✅ Cu RAG

```
1 # ✅ Cu RAG – bazat pe documente reale
2
3 question = "Ce spune contractul despre reziliere?"
4
5 # 1. Caută chunks relevante în pgvector
6 results = rag.search(question, top_k=3)
7
8 # 2. Construiește contextul citabil
9 context = "\n\n".join(
10     f"[{chunk.document.filename}] {chunk.content}"
11     for chunk, _ in results
12 )
13
```

Flow RAG complet:



Avantaj cheie: agentul citează surse reale, nu inventează → reduce hallucinations cu 80-90%

AnalystAgent — Integration Pattern



```
1  ...
2
3  def _build_context(self, results: list[tuple[DocumentChunk, float]]) -> str:
4      # chunks → context string cu relevance scores
5      ...
6
7  def _build_prompt(self, question: str, context: str) -> str:
8      # prompt cu instrucțiuni: citează / „nu știu” / concis
9      ...
10
11 def _extract_sources(self, results: list[tuple[DocumentChunk, float]]) -> list[dict]:
12     # chunks → metadata pentru citare
13     ...
14
15 def _calculate_confidence(self, results: list[tuple[DocumentChunk, float]]) -> float:
16     # avg similarity score → 0.0 - 1.0
17     ...
18
```

`answer_question()`

Entry point principal
Orchestrate toate etapele

`_build_context()`

Chunks → context string
Cu relevance scores

`_build_prompt()`

Construiește prompt
Cu instrucțiuni clare

`_extract_sources()`

Chunks → metadata
Pentru citare surse

`_calculate_confidence()`

Avg similarity score
0.0 - 1.0

_build_context() + _build_prompt()

```
1 class AnalystAgent:
2     # ... __init__, client, answer_question (vezi slide 74)
3
4     def _build_context(
5         self,
6         results: list[tuple[DocumentChunk, float]],
7     ) -> str:
8         # chunks -> context string, cu score-uri inline pentru transparență
9         parts = []
10        for chunk, score in results:
11            parts.append(
12                f"[{chunk.document.filename} | chunk {chunk.chunk_index} | score {score:.2f}]\n"
13                f"{chunk.content}"
14            )
15        return "\n\n".join(parts)
16
17    def _build_prompt(self, question: str, context: str) -> str:
18        return PROMPT_TEMPLATE.format(context=context, question=question)
19
```

Instrucțiuni prompt esențiale

✓ Răspunde BAZAT pe context

Evită informații externe
necuprinse în documente

✓ Citează clauza/documentul

Source attribution
automată în răspuns

✓ "Nu știu" dacă lipsește

Nu inventa răspunsuri
graceful degradation

✓ Răspuns concis și precis

Nu verbosity
evită padding

Error Handling — 3 Scenarii Critice

No Relevant Context



- Return "Nu am găsit informații relevante."
- Fallback la LLM fără context (cu ⚠ warning)
- Suggest: reformulează întrebarea

```
if not chunks:  
    return "Nu am găsit..."
```

Low Confidence Results



- confidence < 0.3 → warn user
- Afișează scorul: ⚠ (45% confidence)
- Include răspunsul cu disclaimer

```
if confidence < 0.3:  
    return f"⚠ {confidence:.0%}:  
{answer}"
```

Embedding / DB Failure



- Catch EmbeddingGenerationError
- Catch DatabaseError
- Log eroarea + return fallback mesaj

```
except EmbeddingError:  
    return "Eroare la procesare"
```

Observability — Logging, Metrics, LangSmith

Logging Pattern

- `logger.info()` — query primit
- Retrieve time în ms
- Nr chunks + avg similarity
- Generate time în ms

Metrics Collection

- `queries_count` total
- `avg_retrieve_time` (ms)
- `avg_similarity` score
- `no_context_rate` (%)

LangSmith Integration

- `@traceable` decorator
- Auto-log inputs/outputs
- Track LLM calls + costs
- Visualize RAG pipeline

Best Practices | 3 Patterns Esențiale

Context Window Management

1

Limitează context la max_tokens pentru a lăsa spațiu întrebării și răspunsului

```
for chunk, score in chunks:
    chunk_tokens = len(chunk.content) // 4
    if total_tokens + chunk_tokens > max_tokens:
        break # Stop la limită
```

💡 ~4 chars = 1 token. Lasă 1000+ tokens pentru Q+A

Dynamic top_k

2

Ajustează top_k bazat pe calitatea results
Dacă primele 3 sunt > 0.7 similarity, nu trebuie 10

```
high_quality = [(c,s) for c,s in chunks if s > 0.7]
if len(high_quality) >= 3:
    return generate(question, high_quality[:3])
else:
    return generate(question, chunks[:5])
```

💡 Previne context dilution cu chunks irelevante

Source Attribution

3

Citează sursele în fiecare răspuns
Buildează încrederea utilizatorului

```
sources_text = "\n\nSurse:\n"
for chunk, score in chunks:
    sources_text += (
        f"- {chunk.document.filename}, "
        f" chunk {chunk.chunk_index} "
        f" ({score:.0%})\n"
    )
```

💡 Permite utilizatorului să verifice răspunsul

Integration Checklist — Înainte de Production

Error Handling

- ☐ Graceful degradation când RAG failuiește
- ☐ No context → mesaj clar, nu eroare
- ☐ Low confidence → warn user

Performance

- ☐ Embedding cache pentru queries repetate
- ☐ Context window management (max_tokens)
- ☐ Dynamic top_k bazat pe similarity

Observability

- ☐ Logging: retrieve time, chunks count, similarity
- ☐ Metrics: no_context_rate, avg_retrieve_time
- ☐ LangSmith / LangFuse pentru traces

Quality

- ☐ Source attribution în fiecare răspuns
- ☐ Confidence score afișat utilizatorului
- ☐ Testing cu queries diverse (edge cases)

Temă: Document Analyst cu RAG

Construiește pipeline-ul de extracție + storage cu pgvector și conectează-l la agentul din L1-L2

Flow așteptat:

```

1 # pipeline.py – load → chunk → extract → store
2 def process(file: Path) -> Document:
3     docs = load(file)    # registry pe extensie
4     chunks = split(docs, 800, 100)
5     data = llm.with_structured_output(Invoice)
6     emb = embed(chunks)  # sentence-transformers
7     return repo.save(data, chunks, emb)
8
9 # agent.py – tool RAG conectat la LLM-ul din L1-L2
10 @tool
11 def search_documents(query: str) -> str:
12     return rag.search(query, top_k=3)
13
14 agent.invoke("Ce clauze de reziliere avem?")

```

Ce trebuie făcut:

- 1 Extraction Pipeline (L3)**
 - Loader cu registry pentru PDF / DOCX / TXT
 - Chunking + schemă Pydantic (factură sau contract)
 - Pipeline: load → chunk → extract + salvare JSON
- 2 PostgreSQL + Repository Pattern (L4)**
 - Postgres cu pgvector via Docker + Alembic migrations
 - Modele Document + DocumentChunk (one-to-many)
 - Repository pentru CRUD și transaction management
- 3 RAG cu Embeddings (L4)**
 - Embeddings cu sentence-transformers per chunk
 - Similarity search cu cosine distance + HNSW index
 - RAGService.search(query, top_k) funcțional
- 4 Conectare la Agentul din L1-L2**
 - Wrap RAG ca LangChain tool: search_documents()
 - Adaugă tool-ul la agentul existent
 - Demo: agent răspunde pe baza documentelor încărcate